Tejiendo redes: arquitectura de software entre hilos y nodos

David Ricardo Amaya Rojas

Integrante y rol: arquitecto de software, desarrollador y evaluador

Tecnológica del Oriente

Arquitectura de Software 20262 - N6 Gr A · Unidad 2

Leydi Johana Polo Amador

10 de noviembre de 2026

Video de presentación: https://youtu.be/yhVhYCH01MI

Tabla de contenido

Introducción **3**
Objetivo general 4
Objetivos específicos 4
Descripción del sistema distribuido **4**
Tipo de sistema 4
Componentes funcionales 5
Supuestos de diseño 5
Herramientas seleccionadas 6
Análisis y planificación del proyecto **7**
Definición de requisitos 7
Relevancia de la comunicación entre nodos e hilos 7
Metas del sistema 8
Estrategia del equipo y asignación de tareas 8
Cronograma 9
Modelado UML **9**
Diagrama de componentes 9
Diagrama de secuencia 10
Diagrama de despliegue 11
Selección y justificación de patrones de diseño **12**
Proxy 14
Mediator 14
Observer distribuido 14
Strategy 14
Alternativas consideradas 14
Implementación y simulación del prototipo distribuido **14**
Estructura del prototipo 14
Comunicación entre nodos 15
Manejo de hilos 16
Tolerancia a fallos 16
Simulación del entorno distribuido 16
Pruebas automáticas 16
Recursos del proyecto 17
Evaluación del desempeño y ajustes **17**
Metodología 17
Escalabilidad 17
Robustez 19
Cuellos de botella detectados 20
Ajustes realizados 20
Limitaciones 21
Conclusiones **22**
Referencias **23**

Tejiendo redes: arquitectura de software entre hilos y nodos

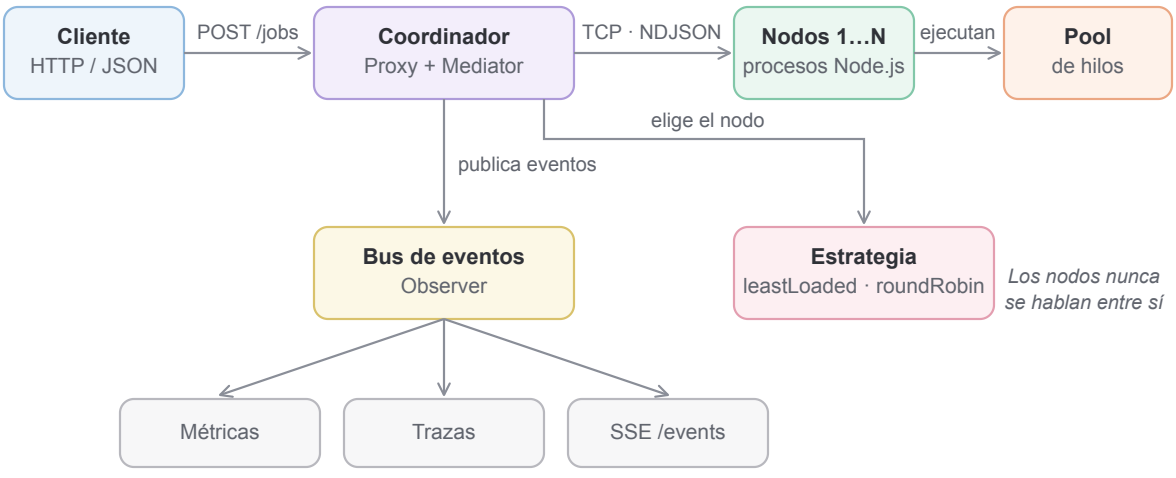
Introducción

Las aplicaciones modernas rara vez se ejecutan en un único proceso. Reparten el trabajo entre varios nodos conectados por una red y, dentro de cada nodo, entre varios hilos, con el fin de atender más carga, tolerar fallos y aprovechar todo el hardware disponible (Tanenbaum y Van Steen, 2017). Diseñar un sistema así exige decisiones arquitectónicas explícitas —cómo se comunican los componentes, quién coordina, qué ocurre cuando un nodo falla— que se comunican y se evalúan mejor con modelos UML y patrones de diseño reconocidos (Bass et al., 2021; Gamma et al., 1994).

Este informe documenta el diseño, la implementación y la evaluación de «Tejiendo redes», un sistema distribuido de procesamiento de tareas intensivas en CPU. Un coordinador recibe las tareas por HTTP y las reparte entre nodos —procesos independientes conectados por TCP— que las ejecutan en pools de hilos. La Figura 1 resume la solución. El propósito de diseñar la arquitectura antes de programar fue decidir con criterios verificables, y no por intuición, cómo se comunican nodos e hilos, qué patrones reducen el acoplamiento y qué mecanismos garantizan que el sistema siga funcionando cuando algo falla.

Figura 1

Mapa conceptual del sistema distribuido «Tejiendo redes»



Nota. Elaboración propia. Las flechas indican la dirección de la comunicación; los nodos solo hablan con el coordinador.

Objetivo general

Diseñar, modelar y evaluar una arquitectura de software distribuida para el procesamiento paralelo de tareas, aplicando especificaciones UML y patrones de diseño, y validarla con un prototipo funcional que demuestre escalabilidad, eficiencia y robustez.

Objetivos específicos

1. Definir el tipo de sistema distribuido, sus componentes, supuestos y herramientas, y planificar el proyecto con requisitos medibles.
2. Modelar la arquitectura con diagramas UML de componentes, secuencia y despliegue.
3. Seleccionar y justificar patrones de diseño pertinentes (Proxy, Mediator, Observer distribuido y Strategy).
4. Implementar un prototipo que evidencie la comunicación entre nodos, el manejo de hilos y la tolerancia a fallos.
5. Evaluar la escalabilidad y la robustez con métricas, identificar cuellos de botella y aplicar ajustes que mejoren el rendimiento.

El documento sigue el orden de la actividad: descripción del sistema, análisis y planificación, modelado UML, patrones de diseño, implementación y simulación, evaluación y ajustes, y conclusiones. El código completo del prototipo está disponible en un repositorio público (Amaya Rojas, 2026).

Descripción del sistema distribuido

Tipo de sistema

Se eligió un sistema distribuido de procesamiento paralelo de tareas con arquitectura maestro–trabajadores. Un coordinador central recibe solicitudes de los clientes, las encola y las asigna a nodos de trabajo; cada nodo es un proceso independiente que ejecuta varias tareas a la vez en su propio pool de hilos. Este tipo de sistema es representativo de servicios reales de transcodificación de video, cifrado masivo o procesos de extracción y transformación de datos.

La tarea de ejemplo es una derivación de clave PBKDF2 con SHA-512, cuyo costo se regula con el parámetro de iteraciones. Se eligió porque es intensiva en CPU, determinista y fácil de verificar, y porque con 150.000 iteraciones dura unos 29 ms en un hilo del equipo de

pruebas, un valor suficientemente grande frente al costo de la comunicación para que el paralelismo se note, y suficientemente pequeño para ejecutar miles de tareas en cada experimento.

Componentes funcionales

La Tabla 1 enumera los componentes, su responsabilidad y el archivo donde se implementan.

Tabla 1

Componentes funcionales del sistema

Componente	Responsabilidad	Archivo
Fachada HTTP (Proxy)	Punto de entrada único: `POST /jobs`, `GET /stats`, `GET /events`.	coordinator.js
Cola global	Retiene las tareas cuando ningún nodo tiene cupo (contrapresión) y prioriza los reintentos.	coordinator.js
Planificador (Strategy)	Elige el nodo que recibe la siguiente tarea: `roundRobin` o `leastLoaded`.	strategies.js
Registro y monitor	Mantiene el estado y la capacidad de cada nodo; expulsa nodos mudos por heartbeat vencido.	coordinator.js
Servidor de enlace (Mediator)	Acepta las conexiones TCP de los nodos y enruta tareas y resultados.	coordinator.js, protocol.js
Bus de eventos (Observer)	Publica `node:joined`, `node:lost`, `job:done`, `job:retry` a los observadores.	event-bus.js
Agente de nodo	Se registra, envía heartbeats, recibe tareas y devuelve resultados; se reconecta solo.	node.js
Pool de hilos	Reparte las tareas entre hilos reutilizables mediante una cola interna.	pool.js
Hilo de trabajo	Ejecuta la tarea CPU-intensiva fuera del hilo principal del nodo.	task-worker.js

Nota. Todos los archivos están en el directorio `src/` del repositorio.

Supuestos de diseño

La Tabla 2 reúne los supuestos con los que se diseñó el sistema y la consecuencia que cada uno tuvo en la arquitectura.

Tabla 2

Supuestos y su consecuencia en el diseño

Supuesto	Consecuencia en el diseño
Los nodos fallan por parada (el proceso muere o deja de responder); no hay nodos maliciosos.	Basta detectar el cierre de conexión o la ausencia de heartbeat; no se requiere consenso bizantino.
Las tareas son idempotentes: ejecutarlas dos veces produce el mismo resultado.	Se adopta entrega «al menos una vez»: ante una caída, la tarea se reasigna sin riesgo.
TCP entrega los mensajes en orden y sin duplicados dentro de una conexión.	El protocolo NDJSON no necesita numeración de mensajes.
El coordinador permanece activo durante la operación.	Se reconoce como punto único de falla y se documenta como mejora futura.
El prototipo corre en un solo equipo físico.	Los nodos son procesos con puertos distintos; no se mide latencia de red real.

Herramientas seleccionadas

El prototipo se implementó en Node.js 25 usando únicamente módulos nativos, de modo que cualquiera pueda reproducirlo sin instalar dependencias. La Tabla 3 resume las herramientas y su justificación.

Tabla 3

Herramientas utilizadas y justificación

Herramienta	Uso	Justificación
Node.js 25 (`net`, `http`)	Servidor TCP de enlace y fachada HTTP	E/S asíncrona de un solo hilo ideal para un coordinador que mayormente enruta mensajes (Node.js, s. f.-b).
`worker_threads`	Hilos de trabajo en cada nodo	Paralelismo real de CPU con comunicación por paso de mensajes, sin memoria compartida mutable (Node.js, s. f.-a).
`child_process`	Lanzar y matar nodos	Simula nodos independientes y permite inyectar fallos con `SIGKILL`.
`node:test`	Pruebas automáticas	Ejecutor de pruebas nativo, sin dependencias (Node.js, s. f.-c).
Git y GitHub	Control de versiones	Historial verificable de commits atribuidos al autor.
SVG, HTML y Chrome headless	Diagramas e informe en PDF	Figuras vectoriales reproducibles y exportación a tamaño carta.

Análisis y planificación del proyecto

Definición de requisitos

Los requisitos se definieron antes de implementar y cada uno tiene un criterio verificable, que se contrasta con los resultados en la sección de evaluación (la Tabla 4).

Tabla 4

Requisitos funcionales y no funcionales

ID	Requisito	Criterio de verificación
RF1	Aceptar tareas por HTTP y devolver el resultado al cliente.	Prueba de clúster: 60 de 60 tareas completadas.
RF2	Repartir tareas entre nodos con una estrategia configurable.	Todos los nodos reciben trabajo; estrategia elegida al arrancar.
RF3	Admitir que los nodos entren y salgan en caliente.	Un nodo reiniciado se reincorpora y vuelve a recibir tareas.
RF4	Reasignar las tareas de un nodo caído (máximo 3 intentos).	Cero tareas perdidas al matar un nodo.
RF5	Exponer el estado y los eventos en vivo.	`GET /stats` y `GET /events` operativos.
RNF1	Escalabilidad: el rendimiento crece al añadir nodos.	Eficiencia de paralelización ≥ 60 % con 4 nodos.
RNF2	Robustez ante la caída de un nodo.	100 % de tareas completadas durante la falla.
RNF3	Eficiencia bajo carga alta.	Latencia p95 ≤ 1 s con 128 clientes concurrentes.
RNF4	Bajo acoplamiento entre nodos.	Ningún nodo conoce a otro nodo.
RNF5	Observabilidad.	Latencias p50/p95/p99 y reparto por nodo.

Relevancia de la comunicación entre nodos e hilos

La comunicación define el comportamiento del sistema en dos niveles. Entre nodos, los mensajes viajan por la red: hay latencia, costo de serialización y fallos parciales, es decir, situaciones en que una parte del sistema falla mientras el resto sigue operando (Kleppmann, 2017). Por eso el enlace usa un protocolo simple, de mensajes JSON delimitados por saltos de línea, y un mecanismo de heartbeat que distingue un nodo lento de uno caído.

Dentro de un nodo, los hilos podrían compartir memoria, pero hacerlo exige sincronización y es una fuente clásica de errores. Los *worker threads* de Node.js se comunican

por paso de mensajes (Node.js, s. f.-a), lo que elimina las condiciones de carrera sobre el estado de las tareas. Además, el trabajo intensivo nunca se ejecuta en el hilo principal del nodo: este debe quedar libre para atender el socket, responder al heartbeat y recibir nuevas tareas.

Dos resultados teóricos orientan la evaluación. La ley de Little (1961) relaciona la concurrencia, el rendimiento y la latencia ($L = \lambda W$), y sirve para verificar que las mediciones son coherentes. La ley de Amdahl (1967) recuerda que la parte secuencial del sistema —aquí, el coordinador— limita la aceleración máxima que se obtiene al añadir recursos.

Metas del sistema

Las metas se derivan de los requisitos: (a) escalar casi linealmente al añadir nodos e hilos; (b) no perder ninguna tarea cuando un nodo cae y recuperarse al volver; y (c) mantener una latencia acotada bajo carga alta, medida con percentiles (p95 y p99) porque los promedios ocultan la cola de latencias lentas (Dean y Barroso, 2013).

Estrategia del equipo y asignación de tareas

El equipo es unipersonal, por lo que el autor asumió todos los roles (la Tabla 5). Para no perder el rigor de una revisión cruzada, el trabajo se organizó de forma iterativa: modelar, implementar el núcleo mínimo, escribir pruebas del comportamiento crítico, medir y solo entonces optimizar. Los cambios se registraron en commits pequeños y descriptivos.

Tabla 5

Roles y asignación de tareas

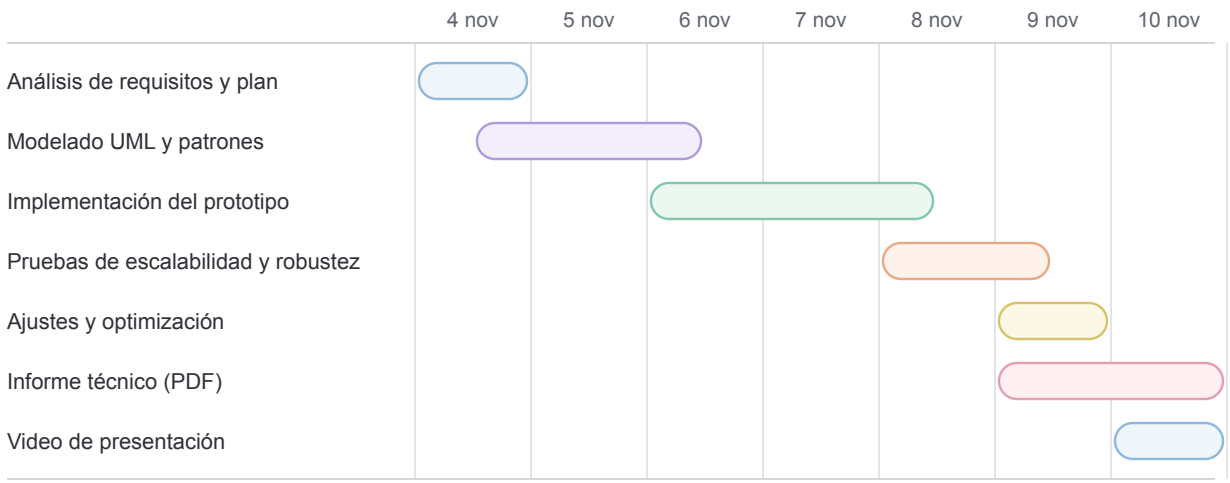
Rol	Responsable	Tareas
Arquitecto de software	David Ricardo Amaya Rojas	Requisitos, diagramas UML, selección de patrones.
Desarrollador	David Ricardo Amaya Rojas	Coordinador, nodos, pool de hilos, protocolo.
Evaluador	David Ricardo Amaya Rojas	Pruebas automáticas, pruebas de carga, análisis de métricas.
Documentación y video	David Ricardo Amaya Rojas	Informe técnico, repositorio y video de presentación.

Cronograma

El cronograma tentativo abarcó siete días, con actividades solapadas (ver la Figura 2).

Figura 2

Cronograma del proyecto (4 al 10 de noviembre de 2026)



Nota. Elaboración propia. Las barras muestran la duración estimada de cada actividad.

Modelado UML

La arquitectura se especificó con UML 2.5.1 (Object Management Group, 2017) mediante tres vistas complementarias: la estructura lógica (componentes), el comportamiento en el tiempo (secuencia) y la distribución física (despliegue) (Fowler, 2004).

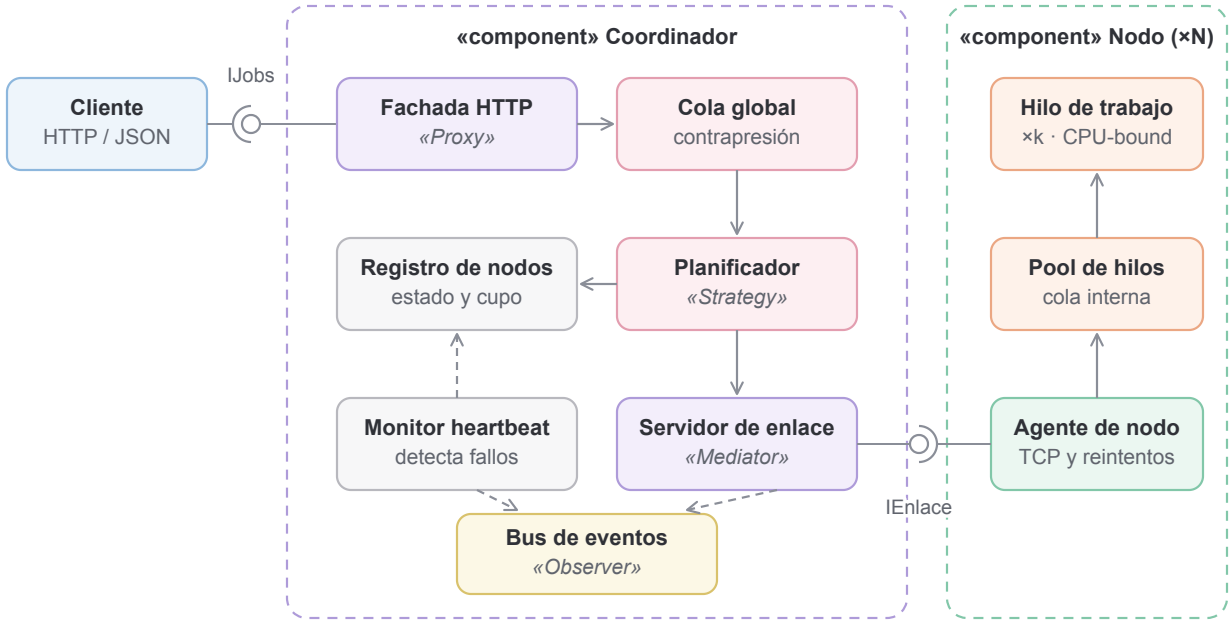
Diagrama de componentes

La Figura 3 muestra dos componentes principales: el coordinador y el nodo de trabajo. Se relacionan mediante dos interfaces: *IJobs*, que el coordinador ofrece a los clientes por HTTP, e *IEnlace*, que el servidor de enlace ofrece a los nodos por TCP. Los nodos son una réplica del mismo componente, de modo que añadir capacidad solo requiere lanzar otra instancia.

Dentro del coordinador, la fachada HTTP entrega cada solicitud a la cola global; el planificador consulta el registro de nodos para elegir un destino y el servidor de enlace envía la tarea. El monitor de heartbeat y el servidor de enlace publican hechos relevantes en el bus de eventos, sin conocer a sus observadores. Dentro del nodo, el agente entrega las tareas al pool, que las distribuye entre los hilos de trabajo.

Figura 3

Diagrama de componentes UML del sistema



Nota. Elaboración propia con notación UML 2.5.1. Las flechas continuas indican dependencia funcional; las punteadas, publicación de eventos o supervisión.

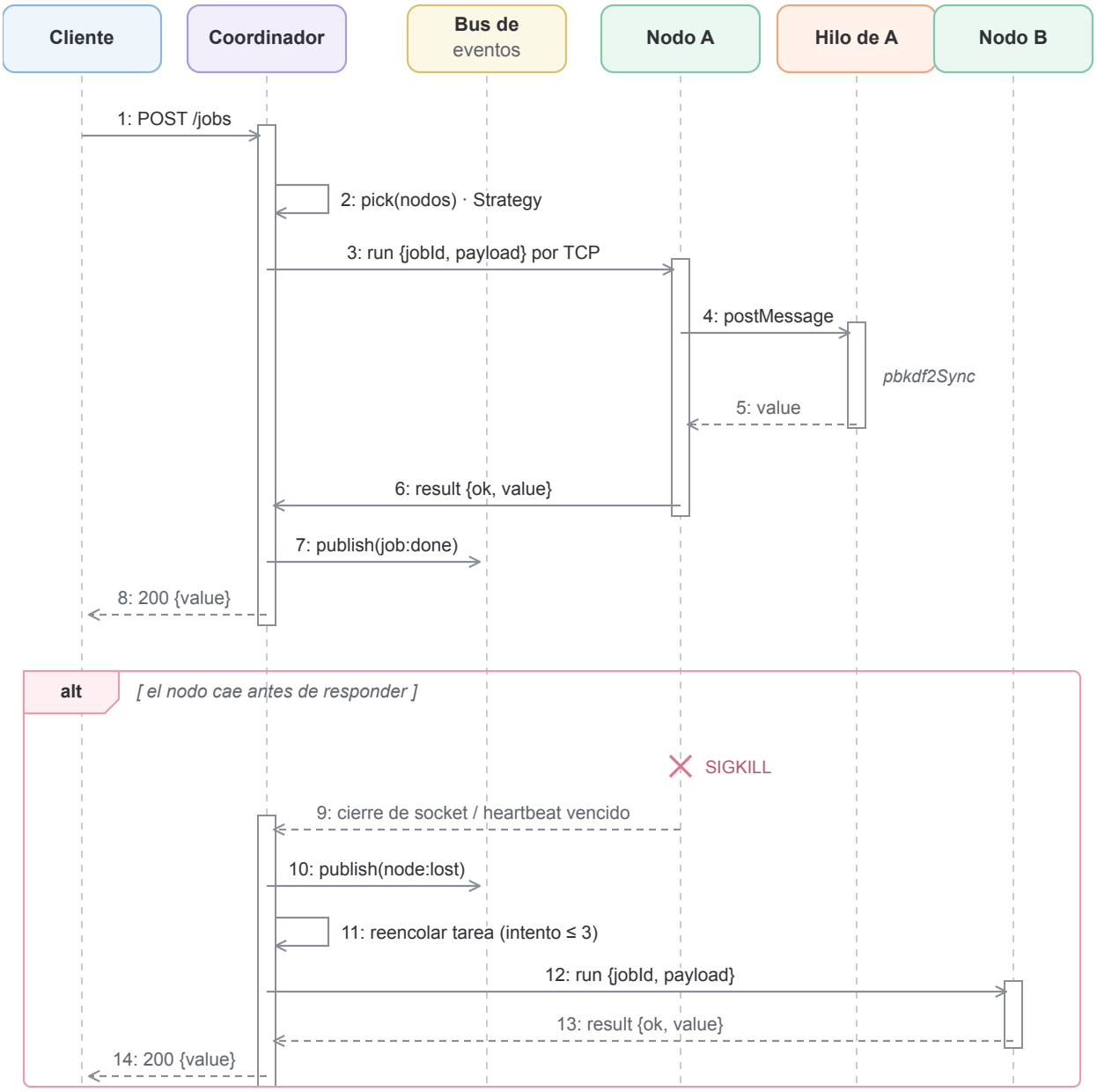
Diagrama de secuencia

La Figura 4 representa la interacción entre nodos e hilos para una tarea. El cliente envía `POST /jobs` (1); el coordinador elige el nodo con la estrategia vigente (2) y le envía el mensaje `run` por TCP (3). El agente del nodo entrega el trabajo a un hilo (4), que ejecuta el cálculo (5) y devuelve el valor. El nodo responde con `result` (6), el coordinador publica el evento `job:done` (7) y responde al cliente (8).

El fragmento *alt* modela el flujo alternativo más importante: si el nodo cae antes de responder (por ejemplo, tras un `SIGKILL`), el coordinador detecta el cierre del socket o el heartbeat vencido (9), publica `node:lost` (10), devuelve la tarea a la cola con prioridad (11) y la envía a otro nodo (12), que responde normalmente (13). El cliente nunca percibe la falla, solo una latencia algo mayor (14).

Figura 4

Diagrama de secuencia UML: ejecución de una tarea y recuperación ante la caída de un nodo



Nota. Elaboración propia. Las líneas punteadas con flecha representan respuestas; el rectángulo estrecho sobre cada línea de vida es la activación del participante.

Diagrama de despliegue

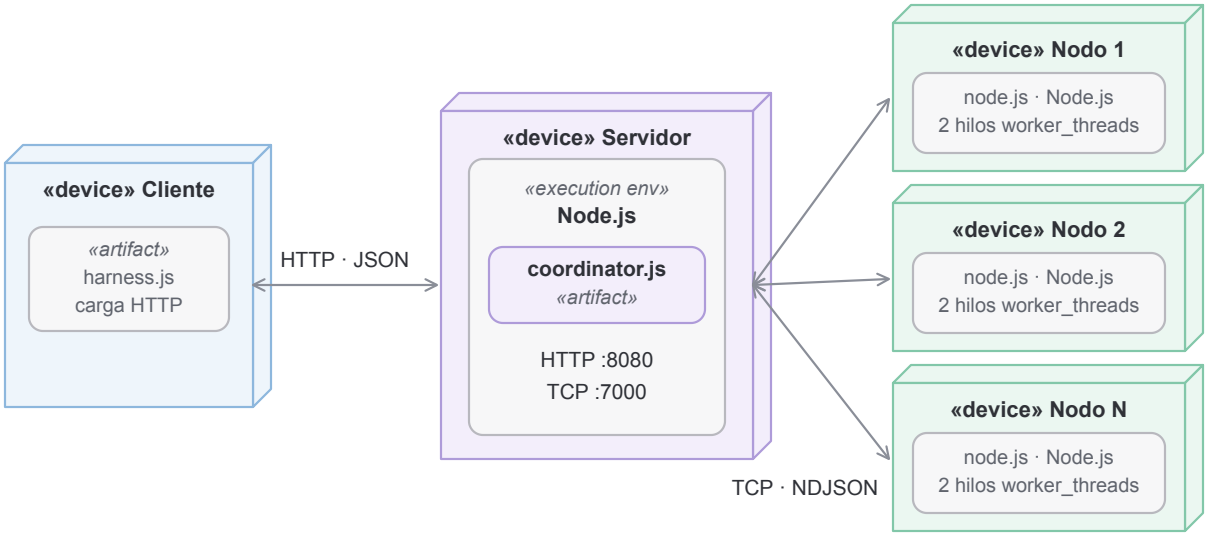
La Figura 5 muestra la distribución física y lógica. Los clientes consumen la API del coordinador por HTTP en el puerto 8080; los nodos se conectan al puerto 7000 del coordinador por TCP. Cada nodo ejecuta un proceso Node.js con su pool de dos hilos. Como la

comunicación siempre la inicia el nodo hacia el coordinador, los nodos pueden estar detrás de un cortafuegos y solo el coordinador necesita puertos abiertos.

En el prototipo, el cliente, el coordinador y los nodos comparten un mismo equipo y se distinguen por proceso y puerto. La arquitectura no cambia al migrar a varios equipos: basta apuntar cada nodo a la dirección del coordinador. El coordinador es el único elemento sin réplica, por lo que constituye un punto único de falla que se discute en las limitaciones.

Figura 5

Diagrama de despliegue UML



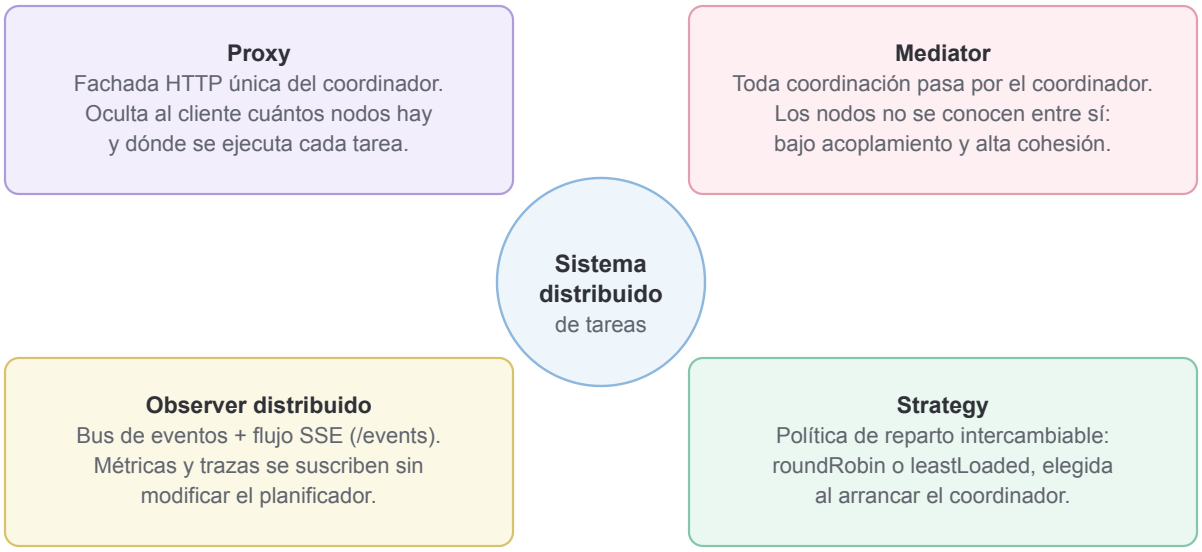
Nota. Elaboración propia. Los nodos adicionales (Nodo N) se añaden sin cambiar el coordinador.

Selección y justificación de patrones de diseño

Los patrones se eligieron por el problema concreto que resuelven en el sistema, siguiendo el catálogo de Gamma et al. (1994). La Figura 6 los ubica en la arquitectura y la Tabla 6 justifica cada uno.

Figura 6

Patrones de diseño aplicados al sistema



Nota. Elaboración propia a partir de la clasificación de Gamma et al. (1994).

Tabla 6

Patrones de diseño: problema, aplicación y justificación

Patrón	Problema que resuelve	Aplicación	Justificación
Proxy	El cliente no debería conocer cuántos nodos existen ni dónde corre cada tarea.	Fachada HTTP del coordinador.	Transparencia de ubicación: se pueden añadir o retirar nodos sin tocar a los clientes.
Mediator	Si los nodos se hablaran entre sí, el acoplamiento crecería con cada nodo (N^2).	Coordinador y servidor de enlace.	Cada nodo conoce un solo interlocutor; las reglas de reparto y recuperación viven en un único lugar.
Observer distribuido	Las métricas y trazas no deben modificar la lógica de planificación.	Bus de eventos y flujo SSE `/events`.	Permite añadir observadores (paneles, alertas) sin cambiar el coordinador; SSE los lleva a clientes remotos (WHATWG, s. f.).
Strategy	La mejor política de reparto depende de la carga y debe poder cambiarse.	`roundRobin` y `leastLoaded`.	Se comparan políticas en la evaluación sin reescribir el planificador.

Proxy

El coordinador actúa como representante del conjunto de nodos: expone una API única y decide por el cliente. Es un proxy de tipo remoto, porque oculta que el trabajo se ejecuta en otros procesos, y también protector, porque los nodos nunca quedan expuestos directamente.

Mediator

Toda interacción entre nodos pasa por el coordinador, que centraliza el registro, la asignación, los reintentos y la expulsión de nodos caídos. Esto reduce las conexiones de $N(N-1)/2$ a N y simplifica el razonamiento sobre el estado global, a cambio de concentrar la responsabilidad en un componente (riesgo que se discute más adelante).

Observer distribuido

El bus de eventos desacopla a quien produce un hecho de quien lo consume. Los nodos también pueden publicar eventos que el coordinador reenvía. El flujo `GET /events` usa *Server-Sent Events* para que un observador remoto reciba los hechos en tiempo real por una conexión HTTP común.

Strategy

La política de reparto es un objeto intercambiable con una única operación, `pick(nodos)`. Con `roundRobin` se rota ciegamente entre nodos; con `leastLoaded` se elige el nodo con menor ocupación relativa (tareas en vuelo sobre capacidad), lo que compensa nodos con distinto número de hilos o más lentos.

Alternativas consideradas

Se evaluó un intermediario de mensajes externo (*message broker*), como describen Hohpe y Woolf (2003), pero se descartó para evitar una dependencia de infraestructura y mantener el prototipo autocontenido; el coordinador cumple ese rol. Se pospuso el patrón *Circuit Breaker* (Nygard, 2018): su utilidad aparece cuando hay llamadas remotas a servicios de terceros, y aquí la protección equivalente es la contrapresión y el límite de reintentos.

Implementación y simulación del prototipo distribuido

Estructura del prototipo

El prototipo consta de aproximadamente 500 líneas de JavaScript sin dependencias externas. El corazón es el método de planificación del coordinador, que despacha tareas

mientras exista un nodo con cupo; si no lo hay, la tarea espera en la cola (la Figura 7). Esa espera es la contrapresión: el sistema se protege en lugar de saturar a los nodos.

Figura 7

Fragmento del planificador del coordinador

```
#pump() {
  while (this.queue.length) {
    const target = this.strategy.pick([...this.nodes.values()]);
    if (!target) return;              // sin cupo: contrapresión
    const job = this.queue.shift();
    job.attempts++;
    target.inflight++;
    target.jobs.add(job.id);
    send(target.socket, { t: 'run', jobId: job.id, payload: job.payload });
  }
}
```

Nota. Extracto de `src/coordinator.js`. La estrategia es intercambiable.

Comunicación entre nodos

Coordinador y nodos intercambian mensajes JSON delimitados por saltos de línea sobre una conexión TCP persistente que inicia el nodo (la Tabla 7). Un módulo de lectura reensambla los fragmentos TCP y entrega solo mensajes completos; las líneas corruptas se descartan sin afectar la conexión.

Tabla 7

Mensajes del protocolo de enlace

Mensaje	Dirección	Contenido
`register`	Nodo → coordinador	Identificador del nodo y número de hilos; el coordinador calcula su capacidad.
`hb`	Nodo → coordinador	Latido cada 500 ms con las tareas en curso.
`run`	Coordinador → nodo	Identificador de la tarea y su carga útil.
`result`	Nodo → coordinador	Resultado (o error) y tiempo de ejecución.

Manejo de hilos

Cada nodo crea al arrancar un pool de hilos de tamaño fijo. Las tareas llegan a una cola interna y se asignan al primer hilo libre; el resultado regresa al hilo principal por mensaje. Para medir el beneficio del pool se implementó también un modo de línea base, `perTask`, que crea un hilo nuevo por cada tarea y lo destruye al terminar.

Tolerancia a fallos

El coordinador detecta una falla por dos vías: el cierre de la conexión TCP (inmediato, por ejemplo cuando el proceso muere) y el vencimiento del heartbeat (para nodos que se cuelgan sin cerrar el socket). En ambos casos expulsa al nodo, publica `node:lost` y devuelve sus tareas en vuelo a la cola, con prioridad, hasta un máximo de tres intentos por tarea. Los nodos se reconectan solos cada 300 ms, de modo que un nodo reiniciado se reincorpora sin intervención.

Simulación del entorno distribuido

El entorno se simula en un equipo con procesos reales: el coordinador corre en el proceso de la prueba y cada nodo es un proceso hijo lanzado con `child_process`. Un generador de carga de ciclo cerrado mantiene un número fijo de clientes HTTP, cada uno de los cuales envía una tarea, espera la respuesta y envía la siguiente. Las fallas se inyectan con `SIGKILL`, que finaliza el proceso sin darle oportunidad de despedirse, como ocurriría con una caída real.

Pruebas automáticas

Ocho pruebas con el ejecutor nativo de Node.js verifican el comportamiento crítico: la selección de las dos estrategias, el aislamiento de observadores defectuosos, el reparto efectivo entre nodos, la caída de un nodo sin pérdida de tareas, la expulsión de un nodo mudo por heartbeat vencido y el rechazo de una tarea inválida tras agotar los reintentos. Esta última prueba descubrió un defecto real: una tarea inválida lanzaba una excepción dentro del hilo y lo destruía, dejando el pool sin capacidad. Se corrigió devolviendo el error como dato y conservando el hilo.

Recursos del proyecto

Código fuente: https://github.com/KIWITECK/tejiendo-redes. El historial contiene todos los commits atribuidos al autor. Los comandos `npm run demo`, `npm test` y `npm run bench` ejecutan la demostración, las pruebas y la evaluación completa. Video de presentación (YouTube, no listado, 13 min 34 s): https://youtu.be/yhVhYCH01MI.

Evaluación del desempeño y ajustes

Metodología

Las mediciones se realizaron en un equipo con procesador Apple M4 (10 núcleos: 4 de rendimiento y 6 de eficiencia) y 16 GB de memoria, con Node.js 25.6. Cada tarea ejecuta 150.000 iteraciones de PBKDF2 (unos 29 ms en un hilo). Cada configuración se midió tres veces, tras un calentamiento de 40 tareas, y se reporta la mediana. Las métricas son el rendimiento (tareas por segundo) y los percentiles p50, p95 y p99 de la latencia de extremo a extremo.

Escalabilidad

La Tabla 8 y la Tabla 9 muestran cómo crece el rendimiento al añadir nodos y al añadir hilos, respectivamente. La aceleración es el cociente entre el rendimiento y el de la configuración mínima, y la eficiencia es la aceleración dividida entre el factor de recursos añadidos.

Tabla 8

Escalabilidad al añadir nodos (2 hilos por nodo, 64 clientes, 600 tareas)

Nodos	Tareas/s	Aceleración	Eficiencia	p50 (ms)	p95 (ms)
1	68,7	1,00	100 %	927	974
2	125,5	1,83	91 %	496	581
3	155,2	2,26	75 %	404	452
4	187,1	2,72	68 %	338	358

Nota. Mediana de tres ejecuciones. No hubo tareas fallidas en ninguna configuración.

Tabla 9

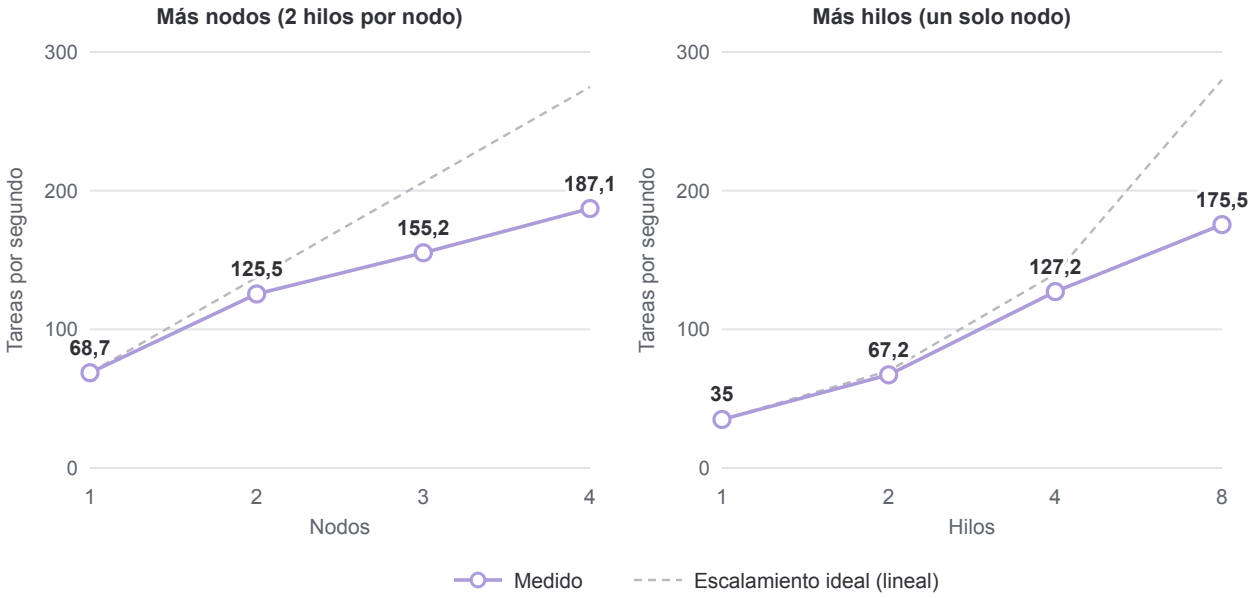
Escalabilidad al añadir hilos (un nodo, 64 clientes, 400 tareas)

Hilos	Tareas/s	Aceleración	Eficiencia	p50 (ms)	p95 (ms)
1	35,0	1,00	100 %	1826	1836
2	67,2	1,92	96 %	940	983
4	127,2	3,63	91 %	498	509
8	175,5	5,01	63 %	362	384

Nota. Mediana de tres ejecuciones. No hubo tareas fallidas en ninguna configuración.

Figura 8

Rendimiento medido frente al escalamiento ideal



Nota. Elaboración propia con los datos de las Tablas 8 y 9. La línea punteada es la extrapolación lineal del rendimiento con la configuración mínima.

El sistema cumple la meta RNF1: con cuatro nodos el rendimiento pasa de 68,7 a 187,1 tareas/s (aceleración de 2,72 y eficiencia de 68 %). En un solo nodo la escala es casi lineal hasta cuatro hilos (eficiencia de 91 %) y cae a 63 % con ocho. La explicación más probable no está en la arquitectura sino en el equipo: ocho hilos de trabajo, el coordinador, el generador de carga y el sistema operativo compiten por solo diez núcleos, seis de ellos de eficiencia, y la porción secuencial del coordinador impone el límite descrito por Amdahl (1967).

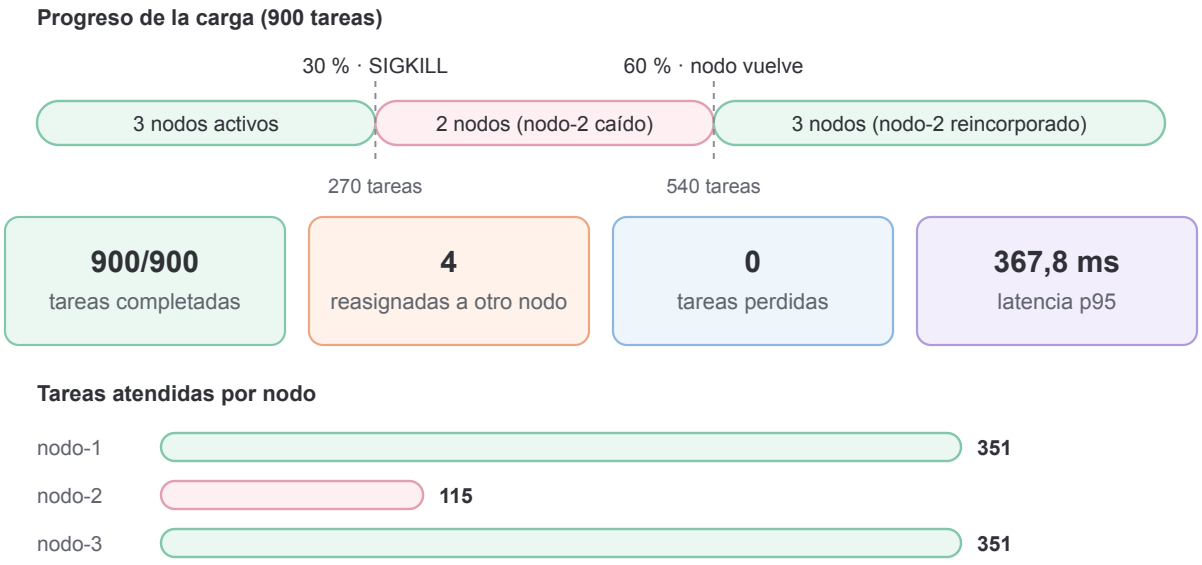
Dos comprobaciones respaldan los datos. Primero, la ley de Little (1961) predice, con 64 clientes y 68,7 tareas/s en un nodo de dos hilos, una latencia media de 64/68,7 = 0,93 s, prácticamente igual a la mediana medida (927 ms). Segundo, cuatro nodos de dos hilos rinden 7 % más que un nodo de ocho hilos, lo que indica que el costo de la comunicación TCP entre procesos es pequeño frente a los 29 ms de cómputo por tarea.

Robustez

Con tres nodos de dos hilos y 900 tareas, se envió `SIGKILL` al nodo-2 al completarse el 30 % de la carga y se lanzó un nodo nuevo con el mismo identificador al llegar al 60 % (la Figura 9). Se completaron las 900 tareas sin ninguna pérdida: las 4 tareas que el nodo caído tenía en vuelo fueron reasignadas y terminaron con dos intentos. El coordinador detectó la caída de inmediato (1 ms) porque el sistema operativo cierra el socket al matar el proceso; el caso de un nodo colgado sin cierre de socket lo cubre el heartbeat, verificado con una prueba automática en que el nodo mudo fue expulsado en poco más de un segundo con un tiempo límite de 800 ms.

Figura 9

Resultados de la prueba de robustez con caída y reincorporación de un nodo



Nota. Elaboración propia. Datos de una ejecución con 3 nodos × 2 hilos, 48 clientes y 900 tareas.

Durante la prueba el rendimiento fue de 150,2 tareas/s, similar al de tres nodos sin fallas (155,2 tareas/s), y la latencia p95 se mantuvo en 368 ms. El nodo reincorporado volvió a recibir trabajo de forma automática, con lo que se cumplen RF3, RF4 y RNF2.

Cuellos de botella detectados

La primera versión (V0) se construyó deliberadamente simple y su medición reveló tres cuellos de botella:

1. **Costo de crear un hilo por tarea.** Cada tarea de 29 ms pagaba además el arranque de un hilo, lo que reduce el rendimiento útil de cada núcleo.
2. **Despacho sin límite y reparto ciego.** Con `roundRobin` y sin tope de tareas en vuelo, el coordinador empuja a los nodos toda la carga de inmediato; las colas internas se forman en los nodos y el reparto no corrige desequilibrios, lo que alarga la cola de la distribución de latencias (p99 de 1.437 ms).
3. **Coordinador como punto único.** Todo el tráfico pasa por un solo ciclo de eventos. En las pruebas no llegó a saturarse (el rendimiento siguió creciendo con cuatro nodos), pero es el límite teórico y un punto único de falla.

Ajustes realizados

Se aplicaron dos ajustes sucesivos y se midió cada uno con el mismo escenario (tres nodos de dos hilos y 128 clientes). V1 sustituye la creación de un hilo por tarea por un pool de hilos reutilizables. V2 cambia la estrategia a `leastLoaded` y añade contrapresión, limitando a dos las tareas en vuelo por hilo; el resto espera en la cola global, donde puede asignarse al nodo que primero se libere.

Tabla 10

Efecto de los ajustes sobre el rendimiento y la latencia (3 nodos × 2 hilos, 128 clientes, 900 tareas)

Versión	Tareas/s	p50 (ms)	p95 (ms)	p99 (ms)
V0. Línea base: round-robin, sin límite, un hilo por tarea	132,5	933	1321	1437
V1. V0 + pool de hilos reutilizables	154,9	784	968	981
V2. V1 + menor carga y contrapresión	160,1	795	833	841

Nota. Mediana de tres ejecuciones. Cero tareas fallidas en las tres versiones.

Figura 10

Comparación de las versiones V0, V1 y V2



Nota. Elaboración propia con los datos de la Tabla 10.

El pool de hilos mejoró el rendimiento 17 % y la latencia p95 27 %. La estrategia de menor carga con contrapresión completó la mejora: frente a la línea base, V2 rinde 21 % más y reduce la latencia p95 en 37 % y la p99 en 41 %. Con V2 se cumple RNF3 (p95 de 833 ms, por debajo de 1 s), que la línea base incumplía. La ganancia principal está en la cola de la distribución: en V2 la diferencia entre la mediana y el p99 es de apenas 46 ms, frente a 504 ms en V0, es decir, el sistema se comporta de forma mucho más predecible. La ley de Little vuelve a ser consistente: 128 / 160,1 = 0,80 s, frente a una mediana de 795 ms.

Limitaciones

- Todos los componentes comparten un equipo; no se midió latencia de red real ni pérdida de paquetes, y el generador de carga comparte proceso con el coordinador.

- El coordinador es un punto único de falla y su cola vive en memoria: si cae, se pierden las tareas pendientes. La mejora natural es una cola persistente y un coordinador de respaldo con elección de líder.
- La entrega es «al menos una vez»: si un nodo termina una tarea justo antes de caer y su resultado se pierde, la tarea se ejecuta dos veces. Es aceptable solo porque las tareas son idempotentes.
- La comunicación no está cifrada ni autenticada; un despliegue real requeriría TLS mutuo entre nodos y coordinador.

Conclusiones

El proyecto cumplió su objetivo: se diseñó, modeló, implementó y evaluó un sistema distribuido de procesamiento de tareas con una arquitectura coherente entre los diagramas UML y el código. Los tres diagramas (componentes, secuencia y despliegue) describieron con precisión lo que el prototipo hace y permitieron razonar sobre las fallas antes de provocarlas.

Los patrones elegidos resolvieron problemas concretos y medibles. Mediator y Proxy mantuvieron el acoplamiento bajo y dejaron que los nodos se añadieran o cayeran sin afectar a los clientes; Observer permitió observar el sistema sin tocar su lógica; y Strategy hizo posible comparar políticas de reparto y demostrar con datos que la de menor carga con contrapresión reduce la latencia p95 en 37 %.

La evaluación mostró que el sistema escala (aceleración de 2,72 con cuatro nodos), que tolera la caída de un nodo sin perder tareas (900 de 900 completadas) y que optimizar exige medir: la mejora más importante vino de eliminar el costo de crear hilos y de limitar el despacho, no de añadir recursos.

Como aprendizaje, la contrapresión y los percentiles de latencia resultaron más reveladores que el rendimiento promedio, y las pruebas automáticas de fallos destaparon un defecto que las pruebas felices no habrían encontrado. Como trabajo futuro quedan la persistencia de la cola, la replicación del coordinador con elección de líder, la autenticación entre componentes y la medición en varias máquinas con red real.

Referencias

Amaya Rojas, D. R. (2026). *Tejiendo redes: Prototipo de sistema distribuido con nodos e hilos* [Software]. GitHub. https://github.com/KIWITECK/tejiendo-redes

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the April 18-20, 1967, spring joint computer conference* (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560

Bass, L., Clements, P., y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Dean, J., y Barroso, L. A. (2013). The tail at scale. *Communications of the ACM, 56*(2), 74–80. https://doi.org/10.1145/2408776.2408794

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research, 9*(3), 383–387. https://doi.org/10.1287/opre.9.3.383

Node.js. (s. f.-a). *Worker threads*. https://nodejs.org/api/worker_threads.html

Node.js. (s. f.-b). *Net*. https://nodejs.org/api/net.html

Node.js. (s. f.-c). *Test runner*. https://nodejs.org/api/test.html

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

Object Management Group. (2017). *Unified Modeling Language (UML), versión 2.5.1*. https://www.omg.org/spec/UML/2.5.1/PDF

Tanenbaum, A. S., y Van Steen, M. (2017). *Distributed systems* (3.ª ed.). Maarten van Steen.

WHATWG. (s. f.). *HTML living standard: Server-sent events*. https://html.spec.whatwg.org/multipage/server-sent-events.html